Adding Interactivity with Turbo Frames

We are now going to add new functionality to our app's schedule page. We're going to allow a user to edit concert data inline on the schedule page. When users hit the Edit button, the display will be replaced by a form, and when they hit the Submit button on the form, the updated data is there in place. (We're not going to worry about authentication very much here. If you are using the existing code, the sign-in links work, and you can sign in with the email areader@example.com and the password awesome. If that username doesn't work, the database hasn't been properly seeded, run rails db:seed to get some data.)

Oh, and we're going to do this without writing a single line of JavaScript. Instead, we are going to use Turbo Frames.

A *Turbo Frame* is a custom HTML tag, <turbo-frame>, that wraps part of our page. Turbo Frames capture navigation. Within a Turbo Frame, any link clicks or form submissions will, by default, capture the server response, look for a Turbo Frame in that response with the same ID, and redraw only that Turbo Frame with only the portion of the response contained by that Turbo Frame. To put it another way, Turbo Frames allow a small part of the page to behave the way Turbo Drive treats the entire page.

Let's use our edit in place as an example.

The display of a single line of concert data is managed by a partial at app/views/concerts/_concert.html.erb. To wrap the entire partial, add this line to the top:

```
<%= turbo_frame_tag(dom_id(concert)) do %>
```

and close it with an <% end %> appended to the end. The main argument to the turbo_frame_tag becomes the DOM ID of the resulting element—here we are using the Rails dom_id method to generate an ID based on the class and id of an ActiveRecord object. Further keyword arguments to turbo_frame_tag become attributes of the resulting element.